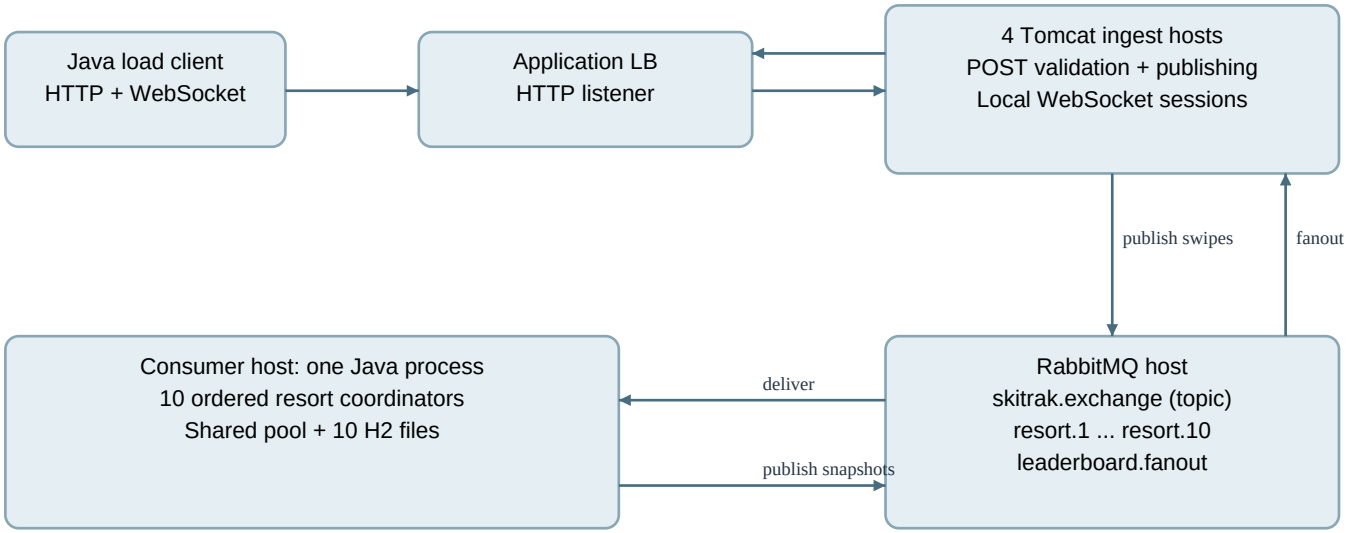
SkiTrak: Architecture and Reliability

Preethi Gajawada | Deployed lab design | 10 November 2026 UTC



Routing and connection locality

The durable topic exchange skitrak.exchange routes key resort.N to durable queue resort.N, for N=1..10. Queue declarations use a configurable TTL (default 86,400,000 ms), maximum length (default 1,000,000), reject-publish overflow and dead-letter exchange skitrak.dead. Persistent messages and publisher confirms support retry after uncertain HTTP outcomes; finite TTL and capacity remain operational limits.

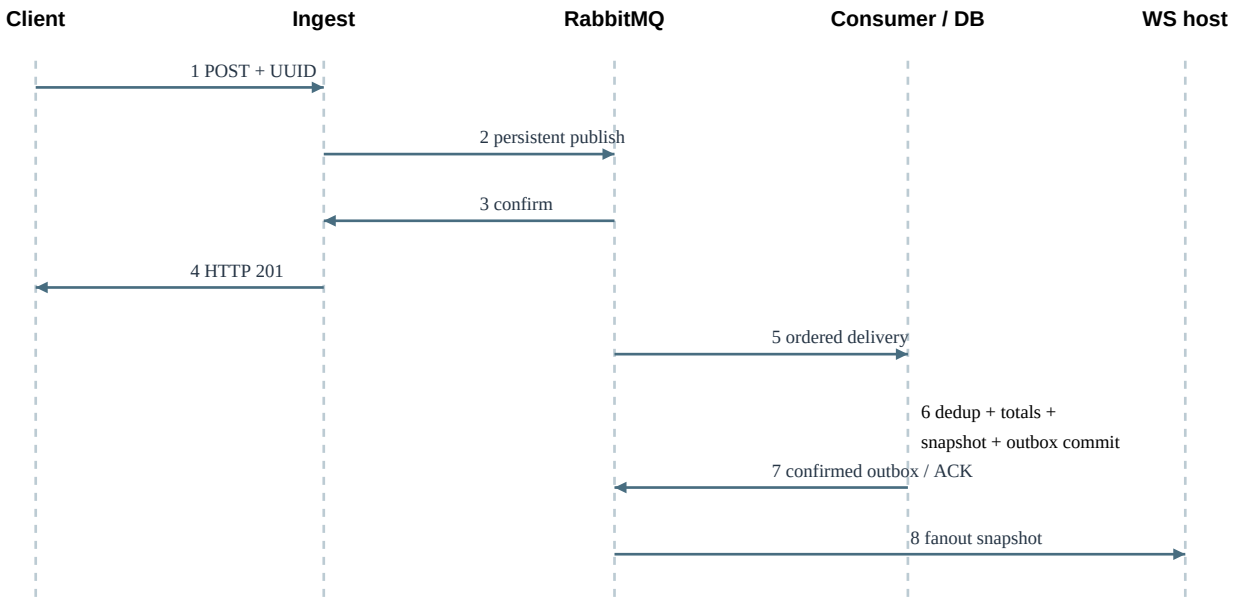
Consumer snapshots publish to durable leaderboard.fanout. Each ingest identity has its own leaderboard.ws.ingest-N queue, so every ingest host receives updates and pushes them to its locally held WebSocket sessions. A retained leaderboard.audit queue also receives updates. This backplane solves connection locality when an HTTP swipe and its subscriber use different ingest hosts. Cookie stickiness cannot by itself move a snapshot to the server holding a socket. Reconnection is required after that server fails.

Ownership, order and aggregation

Each resort has an exclusive queue consumer, one coordinator and one H2 database. A coordinator submits one batch at a time to the shared FIFO executor and waits for completion before reusing its channels. Current batch size/prefetch is 200 with a 20 ms collection wait. Dynamic pool size is 2-10; configured fixed capacities 10/20/40/80 were tested, but active resort tasks cannot exceed ten. HTTP diagnostics use a separate two-thread executor.

Deduplication IDs, skier totals, changed snapshots and an outbox entry commit in one transaction (H2 WRITE_DELAY=0). Same event ID and fingerprint are a duplicate; reuse with different swipe fields is rejected. Top ten is ordered by total vertical descending, then numeric skier ID ascending. Fanout publication is confirmed before input ACKs; replay after failure is safe for aggregation. The design gives idempotent effects with at-least-once delivery, not exactly-once transport.

Swipe to leaderboard sequence



The fanout subscriber sends the snapshot to its local WebSocket sessions. HTTP 201 means broker publication was confirmed; it does not mean aggregation or WebSocket delivery has finished. Snapshot versions change only when the top list changes, so accepted swipes do not imply one WebSocket update each. Subscriber versions and totals are verified independently of transport timestamps.

Load balancer and deployment

One ALB routes HTTP to port 8080 on four t3.micro ingest hosts. Health path /ski-servlet/health uses HTTP 200, interval 30 sec, timeout 5 sec, healthy threshold 2 and unhealthy threshold 3. Routing is round robin; load-balancer cookie stickiness lasts 300 sec. Connection idle timeout was increased from 60 to 300 sec; HTTP client keepalive remains 3,600 sec. Consumer and RabbitMQ each run on a separate t3.small host. AMQP uses the broker private address; diagnostics are accessed through SSH forwarding.

Failure handling and practical limits

Ingest uses one publishing connection per host with bounded channels and sender ownership; HTTP callers wait for their own publish confirmation. Unconfirmed outcomes return 503 and require retry with the same UUID. Resort coordinators reconnect after broker failure. Durable outbox records can be replayed after a crash; a confirm followed by a crash before marking published can duplicate a snapshot. Invalid messages are rejected without requeue. The consumer systemd unit restarts on failure after five seconds; RabbitMQ Docker policy is unless-stopped. Tomcat was manually started after VM restart in these experiments.

Observed recovery: persistent event dedup survived SIGKILL, consumer counts reconciled after an in-flight crash, WebSocket subscribers reconnected after ingest failure, and applications recovered after a graceful broker stop/start. Immediate ALB requests can fail before health detection. The deployment has one broker and one consumer host, and local H2 state: it is not highly available across host or disk loss. Queue depth and consumer lag exceeded targets under heavy load; CPU and I/O measurements support a consumer/storage bottleneck.